PYTHON

Python Programming

From Fundamentals to Interview-Ready:
Core Concepts, OOP, Error Handling & 36 Solved Problems

A Complete Coding-Round Preparation Guide

BASICS • DATA TYPES • FUNCTIONS • CLASSES • EXCEPTIONS • PROBLEM SOLVING

Table of Contents

PART 1 — FUNDAMENTALS

1. Introduction to Python

   What is Python, running code, comments

2. Variables & Input / Output

   Variables, naming, print, input, casting, f-strings

3. Data Types & Built-in Functions

   Numbers, Strings, Lists, Tuples, Sets, Dictionaries, built-ins

PART 2 — CONTROL FLOW & FUNCTIONS

4. Operators & Conditionals

   Operators, if / elif / else, ternary

5. Loops

   for, while, break, continue, nested loops

6. Functions

   Arguments, *args/**kwargs, scope, lambda, recursion

PART 3 — CLASSES & ERROR HANDLING

7. Classes & Objects (OOP)

   __init__, self, 4 pillars, dunder methods, real example

8. Exception Handling

   try / except / else / finally, raise, custom exceptions

PART 4 — PROBLEM SOLVING

9. String Problems
10. Math & Number Problems
11. Logic & Conditional Problems
12. Pattern Printing Problems
13. List & Dictionary Problems
14. Mixed / Interview Warm-up Problems

PART 5 — SOLUTIONS & CODE

15. Answers & Explanations (all 36 problems)

APPENDIX

A. Python Cheat Sheet

B. Coding Round Survival Tips

PART ONE

Fundamentals

> *Before you can solve problems, you need a firm grip on how Python stores and moves data. This part covers the raw materials of every program: values, variables, and the built-in types & functions you will reach for in almost every coding round.*

1. Introduction to Python

What is Python?

Python is a high-level, general-purpose programming language known for its clean, readable syntax. Code often reads almost like English, which is exactly why it is the most common language in coding interviews, data science, automation, and web backends.

It is **interpreted** (runs line by line, no separate compile step), **dynamically typed** (you do not declare variable types), and **beginner-friendly** while still powering large systems at Google, Netflix, and Instagram.

> **Why Python for coding rounds?** Short syntax means less time typing boilerplate and more time solving the actual problem. A solution that takes 30 lines in Java often takes 8 lines in Python.

Your First Program

The traditional first program prints a message to the screen using the `print()` function:

```
print("Hello, World!")
```

Output ▸ `Hello, World!`

Running Python Code

There are three common ways to run Python:

Method	How	Best for
Interactive shell	Type `python` in a terminal	Quick tests
Script file	Save as `program.py`, run `python program.py`	Full programs
Online / IDE	VS Code, PyCharm, Replit, Jupyter	Development & practice

Comments

Comments are notes for humans; Python ignores them when running. Use them to explain *why*, not *what*.

```
# This is a single-line comment
x = 5  # inline comment after code

"""
This is a multi-line comment (technically a string),
often used for longer explanations.
"""
print(x)
```

Output ▸ 5

Anything after `#` on a line is ignored. Triple quotes `"""..."""` span multiple lines and are also used for documentation strings.

How Python Runs Your Code

You write source code → the Python interpreter reads it top to bottom → each statement executes immediately. There is no separate build step, so the feedback loop is fast, which is ideal when practicing many small problems.

2. Variables & Input / Output

Variables

A variable is a named label pointing to a value in memory. You create one simply by assigning with `=`. No type declaration is needed.

```
name = "Aarav"      # a string
age = 21            # an integer
height = 5.9        # a float
is_student = True   # a boolean

print(name, age, height, is_student)
```

Output ▸ `Aarav 21 5.9 True`

Naming Rules

Rule	Valid	Invalid
Start with letter or _	`score` , `_temp`	`2score`
Letters, digits, underscores only	`user_age1`	`user-age`
Case-sensitive	`Age` ≠ `age`	—
No reserved keywords	`total`	`class` , `for`

Convention: use `snake_case` for variables and functions (`total_price`), and descriptive names. `n` is fine for a loop counter; `x1a` for a customer age is not.

Output with print()

```
print("Hello")                # single value
print("Sum is", 3 + 4)        # multiple values, comma-separated
print("A", "B", "C", sep="-")  # custom separator
print("no newline", end=" ")   # control line ending
print("same line")
```

Output ▸
```
Hello
Sum is 7
A-B-C
no newline same line
```

Input from the User

The `input()` function pauses the program and reads a line typed by the user. **It always returns a string**, even if the user types a number.

```
name = input("Enter your name: ")
age = input("Enter your age: ")
print("Hi", name, "- next year you will be", int(age) + 1)
```

> **Common bug:** `input()` gives text. To do math you must convert: `int(age)` or `float(age)`. Forgetting this is the #1 beginner error in interview coding.

Type Casting (Conversion)

```
num_str = "42"
num = int(num_str)     # str -> int  => 42
pi = float("3.14")     # str -> float => 3.14
count = str(100)       # int -> str  => "100"
flag = bool(0)         # 0 -> False, non-zero -> True

print(num + 8, pi, count + "%", flag)
```

Output ▸
```
50 3.14 100% False
```

f-strings (Modern Formatting)

f-strings are the cleanest way to embed variables inside text. Put an `f` before the quote and wrap variables in `{ }`.

```
name = "Riya"
score = 92.5
print(f"{name} scored {score}%")
print(f"Rounded: {score:.0f}")        # format spec: 0 decimals
print(f"2 + 2 = {2 + 2}")             # expressions work too
```

Output ▸
```
Riya scored 92.5%
Rounded: 92
2 + 2 = 4
```

REAL-WORLD USE

REAL-WORLD USE

f-strings build log messages, database queries (carefully), file names, and API responses. Nearly every production Python codebase uses them constantly.

3. Data Types & Built-in Functions

Python groups values into **types**. Knowing each type's built-in methods is what separates a fast interview candidate from a slow one. This is the most important chapter in Part 1 — study it well.

3.1 Numbers: int, float, complex

```
a = 10          # int
b = 3.14        # float
c = 2 + 3j      # complex (rarely needed in interviews)

print(type(a), type(b), type(c))
print(10 / 3)   # true division  -> 3.333...
print(10 // 3)  # floor division -> 3
print(10 % 3)   # modulus        -> 1
print(2 ** 5)   # power          -> 32
```

```
Output ▸ <class 'int'> <class 'float'> <class 'complex'>
3.3333333333333335
3
1
32
```

Useful number functions

Function	What it does	Example → Result
`abs(x)`	Absolute value	`abs(-7)` → 7
`round(x, n)`	Round to n decimals	`round(3.14159, 2)` → 3.14
`pow(x, y)`	x to the power y	`pow(2, 10)` → 1024
`divmod(a, b)`	Quotient & remainder	`divmod(17, 5)` → (3, 2)
`min() / max()`	Smallest / largest	`max(4, 9, 2)` → 9
`sum(iterable)`	Add all items	`sum([1,2,3])` → 6

3.2 Strings

A string is an ordered, **immutable** sequence of characters. Strings appear in almost every coding round, so master their methods.

Indexing & Slicing

```
s = "PYTHON"
#    P  Y  T  H  O  N
#    0  1  2  3  4  5     (forward index)
#   -6 -5 -4 -3 -2 -1     (backward index)

print(s[0])      # P    first char
print(s[-1])     # N    last char
print(s[0:3])    # PYT  slice [start:stop)  stop excluded
print(s[2:])     # THON from index 2 to end
print(s[:4])     # PYTH from start to index 4
print(s[::-1])   # NOHTYP  reverse trick!
print(s[::2])    # PTO  every 2nd char
```

Output ▸
```
P
N
PYT
THON
PYTH
NOHTYP
PTO
```

Interview gold: `s[::-1]` reverses any string in one line. Slicing is `[start : stop : step]` and `stop` is always excluded.

Essential String Methods

Method	Purpose	Example → Result
`.upper()`	Uppercase	`"hi".upper()` → "HI"
`.lower()`	Lowercase	`"HI".lower()` → "hi"
`.strip()`	Remove edge whitespace	`" hi ".strip()` → "hi"
`.split(sep)`	String → list	`"a,b,c".split(",")` → [a,b,c]
`.join(list)`	List → string	`"-".join([a,b])` → "a-b"
`.replace(a,b)`	Replace text	`"cat".replace("c","b")` → "bat"
`.find(x)`	Index of x (-1 if none)	`"hello".find("l")` → 2
`.count(x)`	Count occurrences	`"banana".count("a")` → 3
`.startswith()`	Prefix check	`"file.py".endswith(".py")` → True
`.isdigit()`	All digits?	`"123".isdigit()` → True
`.title()`	Title Case	`"hi there".title()` → "Hi There"

```
sentence = "  the quick brown fox  "
clean = sentence.strip()
words = clean.split()              # ['the','quick','brown','fox']
print(len(words), "words")
print("_".join(words))             # the_quick_brown_fox
print(clean.replace("fox", "dog"))  # the quick brown dog
print(clean.title())               # The Quick Brown Fox
```

Output ▸
```
4 words
the_quick_brown_fox
the quick brown dog
The Quick Brown Fox
```

REAL-WORLD USE

REAL-WORLD USE

`.split()` and `.join()` power CSV parsing, cleaning user input, building URLs, and processing log files. They are the workhorses of text handling.

3.3 Lists

A list is an ordered, **mutable** collection that can hold mixed types. It is the most-used container in Python.

```
fruits = ["apple", "banana", "cherry"]
print(fruits[0])        # apple
print(fruits[-1])       # cherry
print(len(fruits))      # 3

fruits[1] = "mango"     # lists are mutable
print(fruits)           # ['apple', 'mango', 'cherry']
```

Output ▸
```
apple
cherry
3
['apple', 'mango', 'cherry']
```

Essential List Methods

Method	Purpose	Example
`.append(x)`	Add to end	`nums.append(5)`
`.insert(i,x)`	Add at index i	`nums.insert(0, 9)`
`.pop(i)`	Remove & return item	`nums.pop()`
`.remove(x)`	Remove first match	`nums.remove(3)`
`.sort()`	Sort in place	`nums.sort(reverse=True)`
`.reverse()`	Reverse in place	`nums.reverse()`
`.index(x)`	Find position	`nums.index(7)`
`.count(x)`	Count value	`nums.count(2)`
`.extend(lst)`	Add many items	`a.extend(b)`

```
nums = [4, 2, 8, 2, 1]
nums.append(10)         # [4, 2, 8, 2, 1, 10]
nums.sort()             # [1, 2, 2, 4, 8, 10]
print("Max:", max(nums), "Min:", min(nums), "Sum:", sum(nums))
print("Count of 2:", nums.count(2))
```

```
Output ▸ Max: 10 Min: 1 Sum: 27
Count of 2: 2
```

List Comprehension (must-know)

A compact way to build a list in one line. Extremely common in interviews.

```
squares = [x**2 for x in range(1, 6)]        # [1, 4, 9, 16, 25]
evens   = [x for x in range(10) if x % 2 == 0]  # [0, 2, 4, 6, 8]
upper   = [w.upper() for w in ["a", "b", "c"]]  # ['A', 'B', 'C']
print(squares)
print(evens)
print(upper)
```

```
Output ▸ [1, 4, 9, 16, 25]
[0, 2, 4, 6, 8]
['A', 'B', 'C']
```

Pattern: `[expression for item in iterable if condition]` . It replaces a whole for-loop-with-append in one readable line.

3.4 Tuples

A tuple is an ordered but **immutable** collection. Use it for fixed data that should not change — coordinates, RGB colors, database records.

```
point = (3, 4)
print(point[0], point[1])       # 3 4

# Tuple unpacking - very Pythonic
x, y = point
print("x =", x, "y =", y)

# Swap without a temp variable
a, b = 1, 2
a, b = b, a
print(a, b)                     # 2 1
```

Output ▸
```
3 4
x = 3 y = 4
2 1
```

REAL-WORLD USE

REAL-WORLD USE

Functions often return multiple values as a tuple: `return name, age` . Tuple unpacking then reads them cleanly. Immutability also makes tuples usable as dictionary keys.

3.5 Sets

A set is an **unordered collection of unique items**. Perfect for removing duplicates and fast membership tests.

```
nums = [1, 2, 2, 3, 3, 3, 4]
unique = set(nums)              # {1, 2, 3, 4}  duplicates gone
print(unique)
print(3 in unique)              # True (very fast lookup)

a = {1, 2, 3}
b = {2, 3, 4}
print(a | b)   # union        {1, 2, 3, 4}
print(a & b)   # intersection {2, 3}
print(a - b)   # difference   {1}
```

Output ▸
```
{1, 2, 3, 4}
True
{1, 2, 3, 4}
{2, 3}
{1}
```

> **Interview trick:** to remove duplicates from a list instantly: `list(set(my_list))`. To check for any repeated element: `len(set(x)) != len(x)`.

3.6 Dictionaries

A dictionary stores **key : value** pairs. It is Python's most powerful container for lookups — think of it as a real-world dictionary where you look up a word (key) to get its meaning (value).

```
student = {
    "name": "Kabir",
    "age": 20,
    "grades": [88, 92, 79]
}

print(student["name"])          # Kabir
print(student.get("email", "N/A"))  # safe: returns "N/A" if missing

student["age"] = 21             # update
student["city"] = "Pune"        # add new key
print(student.keys())
print(student.values())
```

Output ▸
```
Kabir
N/A
dict_keys(['name', 'age', 'grades', 'city'])
dict_values(['Kabir', 21, [88, 92, 79], 'Pune'])
```

Looping over a dictionary

```
prices = {"pen": 10, "book": 45, "bag": 300}

for item, cost in prices.items():
    print(f"{item} costs Rs.{cost}")

print("Total:", sum(prices.values()))
```

Output ▸
```
pen costs Rs.10
book costs Rs.45
bag costs Rs.300
Total: 355
```

Key dictionary methods

Method	Purpose
`.get(key, default)`	Safe access, no error if missing
`.keys()`	All keys
`.values()`	All values
`.items()`	All key-value pairs
`.pop(key)`	Remove & return value
`.update(d)`	Merge another dict

REAL-WORLD USE

REAL-WORLD USE

Dictionaries model JSON data from APIs, count word frequencies, cache results, and store configuration. The "frequency counter" pattern (counting items with a dict) appears in countless interview problems.

3.7 Universal Built-in Functions

These work across many types and appear everywhere. Memorize them.

Function	Purpose	Example → Result
`len(x)`	Length / size	`len("abc")` → 3
`type(x)`	Type of value	`type(5)` → int
`range(n)`	Sequence 0..n-1	`list(range(3))` → [0,1,2]
`enumerate(x)`	Index + value pairs	see below
`zip(a,b)`	Pair up two lists	see below
`sorted(x)`	Return new sorted list	`sorted([3,1,2])` → [1,2,3]
`map(f,x)`	Apply f to each item	see below
`filter(f,x)`	Keep items where f true	see below

```python
# enumerate: get index AND value together
for i, letter in enumerate(["a", "b", "c"]):
    print(i, letter)

# zip: walk two lists in parallel
names  = ["Amy", "Bob"]
scores = [90, 85]
for n, s in zip(names, scores):
    print(n, s)
```

Output ▸
```
0 a
1 b
2 c
Amy 90
Bob 85
```

```python
# map: transform every element
nums = [1, 2, 3, 4]
doubled = list(map(lambda x: x * 2, nums))   # [2, 4, 6, 8]

# filter: keep matching elements
evens = list(filter(lambda x: x % 2 == 0, nums))  # [2, 4]

# sorted with a key
words = ["banana", "kiwi", "apple"]
by_len = sorted(words, key=len)              # ['kiwi', 'apple', 'banana']
print(doubled)
print(evens)
print(by_len)
```

Output ▸
```
[2, 4, 6, 8]
[2, 4]
['kiwi', 'apple', 'banana']
```

Coding-round tip: `sorted(data, key=...)` handles most "sort by something" questions. `enumerate` avoids the clumsy `range(len(x))` pattern and impresses interviewers.

PART TWO

Control Flow & Functions

> *Data alone does nothing. This part gives your programs the ability to make decisions, repeat work, and be organized into reusable blocks — the logic engine behind every problem you will solve later.*

4. Operators & Conditionals

4.1 Operators

Category	Operators	Example
Arithmetic	`+ - * / // % **`	`7 // 2` → 3
Comparison	`== != > < >= <=`	`5 > 3` → True
Logical	`and or not`	`True and False` → False
Assignment	`= += -= *= //=`	`x += 1`
Membership	`in not in`	`"a" in "cat"` → True
Identity	`is is not`	`x is None`

```
x = 10
print(x > 5 and x < 20)     # True  (both conditions)
print(x > 15 or x == 10)    # True  (at least one)
print(not (x == 10))        # False
print("py" in "python")     # True
print(7 % 2 == 0)           # False (odd number check)
```

Output ▸
```
True
False
True
False
```

Modulus `%` is everywhere in interviews: `n % 2 == 0` checks even, `n % k == 0` checks divisibility, and it wraps values around a range.

4.2 Conditional Statements

Use `if / elif / else` to run different code based on conditions. Indentation (4 spaces) defines the block — Python has no curly braces.

```
marks = 76

if marks >= 90:
    grade = "A"
elif marks >= 75:
    grade = "B"
elif marks >= 60:
    grade = "C"
else:
    grade = "F"

print("Grade:", grade)
```

Output ▸ `Grade: B`

Nested and combined conditions

```
age = 25
income = 40000

if age >= 18:
    if income > 25000:
        print("Loan approved")
    else:
        print("Income too low")
else:
    print("Must be an adult")
```

Output ▸ `Loan approved`

Ternary (one-line if)

```
n = 7
result = "Even" if n % 2 == 0 else "Odd"
print(result)               # Odd

# common in list comprehensions
labels = ["Even" if x % 2 == 0 else "Odd" for x in range(4)]
print(labels)               # ['Even', 'Odd', 'Even', 'Odd']
```

Output ▸
```
Odd
['Even', 'Odd', 'Even', 'Odd']
```

REAL-WORLD USE

REAL-WORLD USE

Conditionals drive every decision in software: validating passwords, routing users by role, applying discounts, choosing what to show on a screen. If-logic is the backbone of business rules.

5. Loops

5.1 The for Loop

Use `for` to iterate over a sequence (list, string, range, dictionary).

```
# Loop over a range
for i in range(1, 6):
    print(i, end=" ")        # 1 2 3 4 5
print()

# Loop over a list
for fruit in ["apple", "banana"]:
    print(fruit)

# range with step
for i in range(0, 11, 2):
    print(i, end=" ")        # 0 2 4 6 8 10
```

```
Output ▸ 1 2 3 4 5
apple
banana
0 2 4 6 8 10
```

`range(start, stop, step)` — `stop` is excluded. `range(5)` gives 0,1,2,3,4.

5.2 The while Loop

Repeats **as long as** a condition stays true. Use when you do not know the number of iterations in advance.

```
count = 5
while count > 0:
    print(count, end=" ")
    count -= 1               # MUST change, or infinite loop!
print("Liftoff!")
```

```
Output ▸ 5 4 3 2 1 Liftoff!
```

Warning: always ensure the while-condition eventually becomes False. Forgetting to update the counter causes an infinite loop — a classic interview mistake.

5.3 break, continue, and loop-else

```
# break: exit the loop immediately
for n in range(1, 10):
    if n == 5:
        break
    print(n, end=" ")        # 1 2 3 4
print()

# continue: skip to next iteration
for n in range(1, 6):
    if n % 2 == 0:
        continue             # skip evens
    print(n, end=" ")        # 1 3 5
```

Output ▸
```
1 2 3 4
1 3 5
```

```
# for-else: else runs only if loop finished WITHOUT break
target = 7
for n in [2, 4, 6, 8]:
    if n == target:
        print("Found")
        break
else:
    print("Not found")       # this runs
```

Output ▸
```
Not found
```

5.4 Nested Loops

A loop inside a loop — essential for grids, tables, and pattern printing (covered heavily in the problem section).

```
for i in range(1, 4):        # outer: rows
    for j in range(1, 4):    # inner: columns
        print(f"{i}x{j}={i*j}", end="  ")
    print()                  # newline after each row
```

Output ▸
```
1x1=1  1x2=2  1x3=3
2x1=2  2x2=4  2x3=6
3x1=3  3x2=6  3x3=9
```

REAL-WORLD USE

REAL-WORLD USE

Loops process every row of a spreadsheet, every user in a database, every pixel in an image, and every item in a shopping cart. Nested loops handle 2D data like game boards and matrices.

6. Functions

A function is a named, reusable block of code. It keeps programs organized, avoids repetition, and makes logic testable. In interviews you are almost always asked to "write a function that…".

6.1 Defining & Calling

```
def greet(name):
    """Return a greeting message."""   # docstring
    return f"Hello, {name}!"

message = greet("Sara")     # call the function
print(message)
```

```
Output ▸ Hello, Sara!
```

`def` defines it, `return` sends a value back. A function without `return` gives back `None`.

6.2 Parameters & Arguments

```
# Positional arguments
def add(a, b):
    return a + b
print(add(3, 4))            # 7

# Default parameter values
def power(base, exp=2):
    return base ** exp
print(power(5))             # 25  (exp defaults to 2)
print(power(5, 3))          # 125

# Keyword arguments (order does not matter)
def profile(name, age):
    return f"{name} is {age}"
print(profile(age=30, name="Dev"))
```

```
Output ▸ 7
25
125
Dev is 30
```

6.3 *args and **kwargs

Accept any number of arguments. `*args` collects extra positional args into a tuple; `**kwargs` collects extra keyword args into a dictionary.

```python
def total(*args):
    return sum(args)
print(total(1, 2, 3, 4))    # 10  (any count)

def describe(**kwargs):
    for key, value in kwargs.items():
        print(f"{key}: {value}")
describe(name="Ana", role="Engineer", city="Delhi")
```

```
Output ▸ 10
name: Ana
role: Engineer
city: Delhi
```

6.4 Scope: Local vs Global

```python
count = 0           # global variable

def increment():
    global count    # tell Python to use the global one
    count += 1

increment()
increment()
print(count)        # 2

def local_demo():
    x = 99          # local, only exists inside this function
    return x
print(local_demo())  # 99
```

```
Output ▸ 2
99
```

> **Rule:** variables created inside a function are *local* and vanish when the function ends. Prefer returning values over using `global` — it is cleaner and less bug-prone.

6.5 Lambda (Anonymous) Functions

A tiny one-line function without a name. Handy with `sorted`, `map`, and `filter`.

```
square = lambda x: x ** 2
print(square(6))            # 36

# Most common real use: as a sort key
people = [("Amy", 25), ("Bob", 19), ("Cara", 32)]
by_age = sorted(people, key=lambda p: p[1])
print(by_age)               # sorted by second element (age)
```

Output ▸ 36
[('Bob', 19), ('Amy', 25), ('Cara', 32)]

6.6 Recursion (Basics)

A function that calls itself. Every recursion needs a **base case** to stop.

```
def factorial(n):
    if n <= 1:           # base case
        return 1
    return n * factorial(n - 1)   # recursive step

print(factorial(5))      # 120  (5*4*3*2*1)
```

Output ▸ 120

REAL-WORLD USE

REAL-WORLD USE

Functions are how real software is built: each feature (login, checkout, search) is a set of functions. Well-named functions make a 1000-line program readable. Interviewers judge you on clean function design.

PART THREE

Classes & Error Handling

> *These two topics separate beginners from job-ready developers. Object-Oriented Programming lets you model real things as code, and Exception Handling keeps your programs from crashing. Both come up constantly in interviews — so this part is deliberately thorough.*

7. Classes & Objects (OOP)

7.1 Why Classes?

Imagine tracking 100 students. Using separate variables (`name1`, `age1`, `name2`…) is chaos. A **class** is a blueprint that bundles related data (attributes) and behavior (methods) into one unit. An **object** is a specific thing built from that blueprint.

> **Analogy:** A class is like a cookie-cutter; objects are the cookies. One blueprint, many objects. The class `Car` defines what a car *is*; `my_car` and `your_car` are individual objects.

7.2 Defining a Class

```
class Dog:
    def __init__(self, name, breed):   # constructor
        self.name = name               # attribute
        self.breed = breed

    def bark(self):                    # method
        return f"{self.name} says Woof!"

# Create objects (instances)
d1 = Dog("Bruno", "Labrador")
d2 = Dog("Coco", "Beagle")

print(d1.name, "-", d1.breed)   # Bruno - Labrador
print(d1.bark())                # Bruno says Woof!
print(d2.bark())                # Coco says Woof!
```

```
Output ▸ Bruno - Labrador
Bruno says Woof!
Coco says Woof!
```

The three key pieces

Piece	What it is
`__init__`	The **constructor** — runs automatically when you create an object. Sets up initial attributes.
`self`	Refers to **the current object**. Every method's first parameter. It is how an object accesses its own data.
attribute	A variable that belongs to the object (`self.name`).

> **Common confusion:** `self` is not typed when calling (`d1.bark()`), Python passes it automatically. You only write `self` when *defining* methods.

7.3 Instance vs Class Variables

```
class Student:
    school = "Delhi Public School"   # CLASS variable - shared by all

    def __init__(self, name):
        self.name = name             # INSTANCE variable - unique per object

s1 = Student("Ravi")
s2 = Student("Meera")
print(s1.name, "-", s1.school)       # Ravi - Delhi Public School
print(s2.name, "-", s2.school)       # Meera - Delhi Public School

Student.school = "New Campus"        # change for everyone
print(s1.school)                     # New Campus
```

Output ▸
```
Ravi - Delhi Public School
Meera - Delhi Public School
New Campus
```

7.4 The Four Pillars of OOP

1. Encapsulation — hiding internal data

Bundle data with the methods that use it, and restrict direct access. A leading underscore `_` signals "private by convention"; double underscore `__` triggers name-mangling for stronger hiding.

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance      # private attribute

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount

    def get_balance(self):
        return self.__balance

acc = BankAccount(1000)
acc.deposit(500)
print(acc.get_balance())              # 1500
# print(acc.__balance)  -> AttributeError (protected)
```

Output ▸
```
1500
```

REAL-WORLD USE

REAL-WORLD USE

Encapsulation stops other code from corrupting an object's state — you can only change a bank balance through `deposit()`, which validates the amount. This is how safe, maintainable software is built.

2. Inheritance — reuse and extend

A child class inherits attributes and methods from a parent, then adds or overrides its own. This avoids duplicating code.

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        return "Some sound"

class Cat(Animal):                    # Cat inherits from Animal
    def speak(self):                  # override
        return f"{self.name} says Meow"

class Cow(Animal):
    def speak(self):
        return f"{self.name} says Moo"

for animal in [Cat("Kitty"), Cow("Ganga")]:
    print(animal.speak())
```

Output ▸
```
Kitty says Meow
Ganga says Moo
```

Using super() to extend the parent

```
class Vehicle:
    def __init__(self, brand):
        self.brand = brand

class Car(Vehicle):
    def __init__(self, brand, model):
        super().__init__(brand)       # call parent constructor
        self.model = model

c = Car("Toyota", "Corolla")
print(c.brand, c.model)               # Toyota Corolla
```

Output ▸
```
Toyota Corolla
```

3. Polymorphism — one interface, many forms

The same method name behaves differently depending on the object. Seen above: `speak()` works on any animal but produces the right sound. This lets you write general code that works with many types.

```
def make_it_speak(animal):
    print(animal.speak())    # works for ANY object with speak()

make_it_speak(Cat("Tom"))
make_it_speak(Cow("Nandini"))
```

```
Output ▸ Tom says Meow
Nandini says Moo
```

4. Abstraction — show essentials, hide complexity

You use `len("hi")` without knowing how it counts characters internally. Classes expose simple methods while hiding messy details inside.

7.5 Special (Dunder) Methods

Methods with double underscores customize how objects behave with built-in operations. `__str__` controls what `print()` shows.

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __str__(self):                # nice printable form
        return f"Point({self.x}, {self.y})"

    def __eq__(self, other):          # define == comparison
        return self.x == other.x and self.y == other.y

p1 = Point(2, 3)
p2 = Point(2, 3)
print(p1)                             # Point(2, 3)   (uses __str__)
print(p1 == p2)                       # True          (uses __eq__)
```

```
Output ▸ Point(2, 3)
True
```

7.6 Complete Real-World Example

A small library system tying it all together — the kind of design question asked in interviews.

```python
class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author
        self.is_borrowed = False

    def __str__(self):
        status = "Borrowed" if self.is_borrowed else "Available"
        return f"'{self.title}' by {self.author} [{status}]"

class Library:
    def __init__(self):
        self.books = []              # list of Book objects

    def add_book(self, book):
        self.books.append(book)

    def borrow(self, title):
        for book in self.books:
            if book.title == title and not book.is_borrowed:
                book.is_borrowed = True
                return f"You borrowed '{title}'"
        return f"'{title}' is not available"

lib = Library()
lib.add_book(Book("Python 101", "A. Sharma"))
lib.add_book(Book("Clean Code", "R. Martin"))

print(lib.borrow("Python 101"))
for b in lib.books:
    print(b)
```

Output ▸
```
You borrowed 'Python 101'
'Python 101' by A. Sharma [Borrowed]
'Clean Code' by R. Martin [Available]
```

REAL-WORLD USE

REAL-WORLD USE

Every app you use is built from classes: a `User`, a `Post`, a `Cart`, an `Order`. OOP models the real world, which is why interviewers love "design a parking lot / library / vending machine" questions.

8. Exception Handling

8.1 Errors vs Exceptions

A **syntax error** means your code is written wrong and will not run at all (e.g. a missing colon). An **exception** is a runtime problem that occurs while a correctly-written program runs — like dividing by zero or reading a missing file. Exception handling lets you catch these gracefully instead of crashing.

```
# Without handling - the program CRASHES here
number = int("hello")   # ValueError: invalid literal for int()
print("This line never runs")
```

> **Why it matters:** a crash in production means a broken app for users. Handling exceptions lets you show a friendly message, retry, or log the issue and keep running.

8.2 try / except

```
try:
    age = int(input("Enter age: "))    # user types "abc"
    print("Next year:", age + 1)
except ValueError:
    print("Please enter a valid number")
```

Output ▸ `Please enter a valid number`

Code that might fail goes in `try`. If an exception occurs, Python jumps straight to the matching `except` block instead of crashing.

8.3 Handling Specific Exceptions

Catch different errors differently. Always prefer specific exceptions over a bare `except`.

```
def divide(a, b):
    try:
        return a / b
    except ZeroDivisionError:
        return "Cannot divide by zero"
    except TypeError:
        return "Both inputs must be numbers"

print(divide(10, 2))       # 5.0
print(divide(10, 0))       # Cannot divide by zero
print(divide(10, "x"))     # Both inputs must be numbers
```

```
Output ▸ 5.0
Cannot divide by zero
Both inputs must be numbers
```

Common built-in exceptions

Exception	Raised when…
`ValueError`	Right type, wrong value (`int("abc")`)
`TypeError`	Wrong type (`"2" + 2`)
`ZeroDivisionError`	Dividing by zero
`IndexError`	List index out of range
`KeyError`	Missing dictionary key
`FileNotFoundError`	Opening a file that does not exist
`AttributeError`	Object has no such attribute/method

8.4 else and finally

```python
try:
    num = int("42")
except ValueError:
    print("Failed to convert")
else:
    print("Success:", num)     # runs ONLY if no exception
finally:
    print("This ALWAYS runs")  # cleanup, runs no matter what
```

```
Output ▸ Success: 42
This ALWAYS runs
```

Block	When it runs
`try`	Always attempted first
`except`	Only if a matching exception occurs
`else`	Only if NO exception occurred
`finally`	Always — used for cleanup (closing files, etc.)

8.5 Raising Your Own Exceptions

Use `raise` to signal an error deliberately when something violates your rules.

```
def set_age(age):
    if age < 0:
        raise ValueError("Age cannot be negative")
    return age

try:
    set_age(-5)
except ValueError as e:
    print("Error:", e)         # access the message with 'as e'
```

Output ▸ `Error: Age cannot be negative`

8.6 Custom Exception Classes

Create your own exception type by inheriting from `Exception`. This makes error handling in large programs clear and specific.

```
class InsufficientFundsError(Exception):
    """Raised when a withdrawal exceeds the balance."""
    pass

class Account:
    def __init__(self, balance):
        self.balance = balance

    def withdraw(self, amount):
        if amount > self.balance:
            raise InsufficientFundsError(
                f"Tried to withdraw {amount}, balance is {self.balance}")
        self.balance -= amount
        return self.balance

acc = Account(500)
try:
    acc.withdraw(800)
except InsufficientFundsError as e:
    print("Transaction blocked:", e)
```

Output ▸ `Transaction blocked: Tried to withdraw 800, balance is 500`

8.7 Best Practices

- Catch **specific** exceptions, not a bare `except:` that hides bugs.
- Keep `try` blocks small — wrap only the risky line.
- Use `finally` (or a `with` block) to release resources.
- Never silently swallow errors with an empty `except: pass` in real code.
- Add a helpful message so debugging later is easy.

REAL-WORLD USE

REAL-WORLD USE

Exception handling is what keeps banking apps, payment gateways, and web servers alive when something unexpected happens — a dropped network call, corrupt data, or a user typing nonsense. Robust error handling is a hallmark of professional code interviewers look for.

PART FOUR

Problem Solving

> *This part is your coding-round gym. Each chapter groups problems by theme, from strings to logic to patterns. Read the statement, then try to solve it yourself before turning to the fully worked solutions in Part Five. Difficulty tags* EASY MEDIUM *help you pace your practice.*

> *How to use this section: attempt every problem on paper or in an editor first. Struggling for a few minutes is where the real learning happens. Only then compare with the reference solution and its explanation.*

9. String Problems

6 problems. Solutions with code and explanations are in Chapter 15 (Part Five), numbered to match.

Problem 9.1. Reverse a string without using any built-in reverse function, then check if it is a palindrome. EASY

Problem 9.2. Count the number of vowels and consonants in a given string. EASY

Problem 9.3. Count the frequency of each character in a string using a dictionary. EASY

Problem 9.4. Check whether two strings are anagrams of each other. MEDIUM

Problem 9.5. Find the first non-repeating character in a string. MEDIUM

Problem 9.6. Capitalize the first letter of every word without using the .title() method. MEDIUM

10. Math & Number Problems

6 problems. Solutions with code and explanations are in Chapter 15 (Part Five), numbered to match.

Problem 10.1. Check whether a given number is prime. EASY

Problem 10.2. Print the Fibonacci series up to n terms. EASY

Problem 10.3. Find the factorial of a number using both a loop and recursion. EASY

Problem 10.4. Check if a number is an Armstrong number (sum of its digits each raised to the power of the number of digits equals the number). MEDIUM

Problem 10.5. Find the GCD (greatest common divisor) of two numbers. MEDIUM

Problem 10.6. Sum the digits of a number until a single digit remains (digital root). MEDIUM

11. Logic & Conditional Problems

6 problems. Solutions with code and explanations are in Chapter 15 (Part Five), numbered to match.

Problem 11.1. Find the largest and smallest number in a list without using max() or min(). EASY

Problem 11.2. Check whether a given year is a leap year. EASY

Problem 11.3. Swap two numbers without using a third (temporary) variable. EASY

Problem 11.4. Remove duplicates from a list while preserving the original order. MEDIUM

Problem 11.5. Count how many times each word appears in a sentence. MEDIUM

Problem 11.6. Given a list of numbers, separate them into two lists: evens and odds. MEDIUM

12. Pattern Printing Problems

6 problems. Solutions with code and explanations are in Chapter 15 (Part Five), numbered to match.

Problem 12.1. Print a right-angled triangle of stars of height n. EASY

Problem 12.2. Print a number pyramid where each row shows numbers 1 to that row number. EASY

Problem 12.3. Print an inverted right-angled triangle of stars. EASY

Problem 12.4. Print a pyramid of stars centered with spaces. MEDIUM

Problem 12.5. Print Pascal's Triangle for n rows. MEDIUM

Problem 12.6. Print a hollow square of stars (border only) of size n. MEDIUM

13. List & Dictionary Problems

6 problems. Solutions with code and explanations are in Chapter 15 (Part Five), numbered to match.

Problem 13.1. Merge two dictionaries, adding values for any shared keys. EASY

Problem 13.2. Find the second largest number in a list. EASY

Problem 13.3. Count the frequency of elements in a list and return the most common one. MEDIUM

Problem 13.4. Flatten a nested list (a list containing lists) into a single list. MEDIUM

Problem 13.5. Given two lists, find the common elements (intersection) and combined unique elements (union). MEDIUM

Problem 13.6. Group a list of words by their first letter into a dictionary. MEDIUM

14. Mixed / Interview Warm-up Problems

6 problems. Solutions with code and explanations are in Chapter 15 (Part Five), numbered to match.

Problem 14.1. Validate whether a password is strong: at least 8 characters, containing an uppercase letter, a lowercase letter, and a digit. MEDIUM

Problem 14.2. Convert a temperature between Celsius and Fahrenheit based on a unit argument. EASY

Problem 14.3. Simulate a simple ATM: given a balance and a list of transactions (positive = deposit, negative = withdrawal), process them and reject withdrawals that exceed the balance. MEDIUM

Problem 14.4. Given a sentence, return the longest word and its length. EASY

Problem 14.5. Check if a string of brackets is balanced (every opening bracket has a matching closing one). MEDIUM

Problem 14.6. Format a list of names into a readable string: 'A, B and C'. MEDIUM

PART FIVE

Solutions & Code

> *Complete, runnable solutions for every problem in Part Four, with the expected output and a clear explanation of the approach. Solution numbers match the problem numbers exactly (e.g. Solution 9.1 answers Problem 9.1).*

15. Answers & Explanations

Chapter 9: String Problems

Solution 9.1

Reverse a string without using any built-in reverse function, then check if it is a palindrome.

```
def reverse_and_check(s):
    reversed_s = ""
    for char in s:
        reversed_s = char + reversed_s   # prepend each char
    is_palindrome = (s == reversed_s)
    return reversed_s, is_palindrome

text = "madam"
rev, palin = reverse_and_check(text)
print("Reversed:", rev)
print("Palindrome:", palin)
```

```
Output ▸ Reversed: madam
Palindrome: True
```

How it works: We build the reverse by prepending each character (`char + reversed_s`). Comparing the original with its reverse tells us if it is a palindrome. The slice trick `s[::-1]` also works, but the manual loop shows the logic.

Solution 9.2

Count the number of vowels and consonants in a given string.

```
def count_letters(s):
    vowels = "aeiou"
    v = c = 0
    for ch in s.lower():
        if ch.isalpha():
            if ch in vowels:
                v += 1
            else:
                c += 1
    return v, c

vowels, cons = count_letters("Hello World")
print("Vowels:", vowels, "Consonants:", cons)
```

```
Output ▸ Vowels: 3 Consonants: 7
```

How it works: We lowercase the string so 'A' and 'a' are treated equally. `isalpha()` skips spaces and punctuation. Each letter is classified as a vowel (in the vowel set) or a consonant.

Solution 9.3

Count the frequency of each character in a string using a dictionary.

```
def char_frequency(s):
    freq = {}
    for ch in s:
        freq[ch] = freq.get(ch, 0) + 1   # count pattern
    return freq

result = char_frequency("banana")
print(result)
```

```
Output ▸ {'b': 1, 'a': 3, 'n': 2}
```

How it works: The **frequency counter** pattern is one of the most tested ideas in interviews. `freq.get(ch, 0)` returns the current count (or 0 if unseen), and we add 1. This same pattern counts words, votes, or any repeated items.

Solution 9.4

Check whether two strings are anagrams of each other.

```
def are_anagrams(a, b):
    a = a.replace(" ", "").lower()
    b = b.replace(" ", "").lower()
    return sorted(a) == sorted(b)

print(are_anagrams("listen", "silent"))   # True
print(are_anagrams("hello", "world"))      # False
```

```
Output ▸ True
False
```

How it works: Two strings are anagrams if they contain the same letters in any order. Sorting both and comparing is the cleanest check — if the sorted characters match, they are anagrams. We strip spaces and lowercase first so phrases work too.

Solution 9.5

Find the first non-repeating character in a string.

```
def first_unique(s):
    freq = {}
    for ch in s:
        freq[ch] = freq.get(ch, 0) + 1
    for ch in s:                    # second pass keeps original order
        if freq[ch] == 1:
            return ch
    return None

print(first_unique("aabbcdd"))     # c
```

```
Output ▸ c
```

How it works: First pass counts every character. Second pass walks the string in order and returns the first character whose count is exactly 1. Two passes over the string keep it efficient and preserve order.

Solution 9.6

Capitalize the first letter of every word without using the .title() method.

```
def capitalize_words(sentence):
    words = sentence.split()
    result = []
    for w in words:
        result.append(w[0].upper() + w[1:])
    return " ".join(result)

print(capitalize_words("the quick brown fox"))
```

```
Output ▸ The Quick Brown Fox
```

How it works: We split into words, then rebuild each word as its first letter uppercased (`w[0].upper()`) plus the rest unchanged (`w[1:]`). `join` stitches the words back with spaces.

Chapter 10: Math & Number Problems

Solution 10.1

Check whether a given number is prime.

```
def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0:
            return False
    return True

for num in [2, 9, 17, 20]:
    print(num, "->", is_prime(num))
```

```
Output ▸ 2 -> True
9 -> False
17 -> True
20 -> False
```

How it works: A prime has no divisors other than 1 and itself. We only check up to the square root (`n ** 0.5`) because any factor larger than the root pairs with a smaller one we already tested — a key optimization interviewers look for.

Solution 10.2

Print the Fibonacci series up to n terms.

```
def fibonacci(n):
    a, b = 0, 1
    series = []
    for _ in range(n):
        series.append(a)
        a, b = b, a + b     # simultaneous update
    return series

print(fibonacci(8))
```

```
Output ▸ [0, 1, 1, 2, 3, 5, 8, 13]
```

How it works: Each number is the sum of the previous two. The tuple assignment `a, b = b, a + b` updates both values at once without a temporary variable — a very Pythonic technique.

Solution 10.3

Find the factorial of a number using both a loop and recursion.

```
def factorial_loop(n):
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result

def factorial_rec(n):
    if n <= 1:
        return 1
    return n * factorial_rec(n - 1)

print(factorial_loop(5))   # 120
print(factorial_rec(5))    # 120
```

```
Output ▸ 120
120
```

How it works: Factorial of n is 1x2x...xn. The loop multiplies iteratively; the recursive version expresses n! = n x (n-1)!. Both are worth knowing — interviewers may ask you to convert between iterative and recursive forms.

Solution 10.4

Check if a number is an Armstrong number (sum of its digits each raised to the power of the number of digits equals the number).

```
def is_armstrong(n):
    digits = str(n)
    power = len(digits)
    total = sum(int(d) ** power for d in digits)
    return total == n

for num in [153, 370, 100]:
    print(num, "->", is_armstrong(num))
```

```
Output ▸ 153 -> True
370 -> True
100 -> False
```

How it works: 153 = 1^3 + 5^3 + 3^3. We convert the number to a string to access each digit, raise each to the power equal to the digit count, and compare the sum to the original number. The generator inside `sum()` keeps it to one line.

Solution 10.5

Find the GCD (greatest common divisor) of two numbers.

```
def gcd(a, b):
    while b != 0:
        a, b = b, a % b     # Euclid's algorithm
    return a

print(gcd(48, 18))   # 6
print(gcd(100, 75))  # 25
```

```
Output ▸ 6
25
```

How it works: Euclid's algorithm: repeatedly replace the pair (a, b) with (b, a mod b) until b becomes 0; the remaining a is the GCD. It is far faster than checking every possible divisor and is a classic interview favorite.

Solution 10.6

Sum the digits of a number until a single digit remains (digital root).

```
def digital_root(n):
    while n >= 10:
        n = sum(int(d) for d in str(n))
    return n

print(digital_root(9875))   # 9875 -> 29 -> 11 -> 2
```

```
Output ▸ 2
```

How it works: We repeatedly add the digits until only one digit is left. Converting the number to a string lets us iterate over digits easily. 9875 -> 29 -> 11 -> 2.

Chapter 11: Logic & Conditional Problems

Solution 11.1

Find the largest and smallest number in a list without using max() or min().

```
def find_extremes(nums):
    largest = smallest = nums[0]
    for n in nums[1:]:
        if n > largest:
            largest = n
        if n < smallest:
            smallest = n
    return largest, smallest

big, small = find_extremes([23, 5, 78, 1, 45])
print("Largest:", big, "Smallest:", small)
```

```
Output ▸ Largest: 78 Smallest: 1
```

How it works: We assume the first element is both largest and smallest, then scan the rest, updating whenever we find something bigger or smaller. This single-pass approach is how `max` / `min` work internally.

Solution 11.2

Check whether a given year is a leap year.

```
def is_leap(year):
    return (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0)

for y in [2000, 1900, 2024, 2023]:
    print(y, "->", is_leap(y))
```

```
Output ▸ 2000 -> True
1900 -> False
2024 -> True
2023 -> False
```

How it works: A leap year is divisible by 4, EXCEPT century years, which must also be divisible by 400. So 2000 is a leap year but 1900 is not. The boolean expression captures both rules in one line.

Solution 11.3

Swap two numbers without using a third (temporary) variable.

```
a, b = 5, 9
print("Before:", a, b)

a, b = b, a      # Pythonic swap

print("After:", a, b)
```

```
Output ▸ Before: 5 9
After: 9 5
```

How it works: Python's tuple assignment evaluates the right side first, then assigns — so `a, b = b, a` swaps cleanly. In other languages you would use a temp variable or arithmetic tricks; Python makes it trivial.

Solution 11.4

Remove duplicates from a list while preserving the original order.

```
def remove_duplicates(items):
    seen = set()
    result = []
    for item in items:
        if item not in seen:
            seen.add(item)
            result.append(item)
    return result

print(remove_duplicates([3, 1, 3, 5, 1, 9, 5]))
```

```
Output ▸ [3, 1, 5, 9]
```

How it works: `list(set(x))` removes duplicates but loses order. To keep order, we track seen items in a set (fast lookups) and only append the first occurrence of each. This order-preserving dedup is a common interview twist.

Solution 11.5

Count how many times each word appears in a sentence.

```
def word_count(sentence):
    counts = {}
    for word in sentence.lower().split():
        counts[word] = counts.get(word, 0) + 1
    return counts

text = "the cat sat on the mat the cat ran"
print(word_count(text))
```

```
Output ▸ {'the': 3, 'cat': 2, 'sat': 1, 'on': 1, 'mat': 1, 'ran': 1}
```

How it works: We lowercase and split into words, then apply the frequency-counter pattern with a dictionary. This is the foundation of search engines, word clouds, and text analytics.

Solution 11.6

Given a list of numbers, separate them into two lists: evens and odds.

```
def split_even_odd(nums):
    evens = [n for n in nums if n % 2 == 0]
    odds  = [n for n in nums if n % 2 != 0]
    return evens, odds

e, o = split_even_odd([1, 2, 3, 4, 5, 6, 7, 8])
print("Evens:", e)
print("Odds:", o)
```

```
Output ▸ Evens: [2, 4, 6, 8]
Odds: [1, 3, 5, 7]
```

How it works: Two list comprehensions filter by the modulus test. This demonstrates how `[x for x in list if condition]` replaces a loop-and-append and reads almost like plain English.

Chapter 12: Pattern Printing Problems

Solution 12.1

Print a right-angled triangle of stars of height n.

```
def star_triangle(n):
    for i in range(1, n + 1):
        print("*" * i)

star_triangle(5)
```

```
Output ▸ *
**
***
****
*****
```

How it works: Row i simply prints i stars. Python's `"*" * i` repeats the character, avoiding an inner loop entirely. Understanding row-vs-column counts is the key to all pattern problems.

Solution 12.2

Print a number pyramid where each row shows numbers 1 to that row number.

```
def number_pyramid(n):
    for i in range(1, n + 1):
        for j in range(1, i + 1):
            print(j, end=" ")
        print()

number_pyramid(4)
```

```
Output ▸ 1
1 2
1 2 3
1 2 3 4
```

How it works: The outer loop controls the row; the inner loop prints numbers 1 up to the current row number. `end=" "` keeps numbers on the same line, and the empty `print()` moves to the next row.

Solution 12.3

Print an inverted right-angled triangle of stars.

```
def inverted_triangle(n):
    for i in range(n, 0, -1):     # count down
        print("*" * i)

inverted_triangle(5)
```

```
Output ▸ *****
****
***
**
*
```

How it works: By counting the range downward (`range(n, 0, -1)`), the first row has the most stars and each subsequent row has one fewer — producing the inverted shape.

Solution 12.4

Print a pyramid of stars centered with spaces.

```
def centered_pyramid(n):
    for i in range(1, n + 1):
        spaces = " " * (n - i)
        stars = "*" * (2 * i - 1)
        print(spaces + stars)

centered_pyramid(4)
```

```
Output ▸    *
   ***
  *****
*******
```

How it works: Each row needs leading spaces (n - i) to center it, then an odd number of stars (2i - 1). Working out these two formulas is the whole trick to centered patterns — a very common interview pattern question.

Solution 12.5

Print Pascal's Triangle for n rows.

```
def pascals_triangle(n):
    row = [1]
    for _ in range(n):
        print(" ".join(map(str, row)))
        # next row: each element is sum of two above
        new_row = [1]
        for i in range(len(row) - 1):
            new_row.append(row[i] + row[i + 1])
        new_row.append(1)
        row = new_row

pascals_triangle(5)
```

```
Output ▸ 1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
```

How it works: Each number is the sum of the two directly above it, with 1s on the edges. We build each new row from the previous one. `map(str, row)` converts numbers to strings so `join` can print them neatly.

Solution 12.6

Print a hollow square of stars (border only) of size n.

```
def hollow_square(n):
    for i in range(n):
        for j in range(n):
            # star only on the border
            if i == 0 or i == n-1 or j == 0 or j == n-1:
                print("*", end="")
            else:
                print(" ", end="")
        print()

hollow_square(4)
```

Output ▸
```
****
*  *
*  *
****
```

How it works: A cell is on the border if it is in the first/last row or first/last column. Everything else is a space. Combining these boundary conditions with `or` produces the hollow effect — great practice for grid logic.

Chapter 13: List & Dictionary Problems

Solution 13.1

Merge two dictionaries, adding values for any shared keys.

```
def merge_add(d1, d2):
    result = d1.copy()
    for key, value in d2.items():
        result[key] = result.get(key, 0) + value
    return result

a = {"apple": 2, "banana": 3}
b = {"banana": 1, "cherry": 5}
print(merge_add(a, b))
```

Output ▸
```
{'apple': 2, 'banana': 4, 'cherry': 5}
```

How it works: We start with a copy of the first dict, then add each entry from the second. `get(key, 0)` handles keys that exist in only one dict. 'banana' appears in both, so its values (3 + 1) combine.

Solution 13.2

Find the second largest number in a list.

```
def second_largest(nums):
    unique = list(set(nums))     # remove duplicates
    unique.sort()
    return unique[-2]            # second from the end

print(second_largest([10, 5, 10, 8, 3]))
```

```
Output ▸ 8
```

How it works: We remove duplicates with `set` (so two 10s do not both count), sort ascending, and take the second-to-last element. Handling duplicates is the subtle part interviewers check for.

Solution 13.3

Count the frequency of elements in a list and return the most common one.

```
def most_common(items):
    freq = {}
    for item in items:
        freq[item] = freq.get(item, 0) + 1
    # find key with max value
    return max(freq, key=freq.get)

votes = ["A", "B", "A", "C", "A", "B"]
print("Winner:", most_common(votes))
```

```
Output ▸ Winner: A
```

How it works: After counting frequencies, `max(freq, key=freq.get)` returns the key with the highest count — `key=freq.get` tells max to compare by count, not by the key itself. This is the vote-counting / mode problem.

Solution 13.4

Flatten a nested list (a list containing lists) into a single list.

```
def flatten(nested):
    result = []
    for element in nested:
        if isinstance(element, list):
            result.extend(flatten(element))   # recurse
        else:
            result.append(element)
    return result

print(flatten([1, [2, 3], [4, [5, 6]], 7]))
```

```
Output ▸ [1, 2, 3, 4, 5, 6, 7]
```

How it works: We loop through each element; if it is itself a list, we recursively flatten it and `extend` the result, otherwise we append the value. `isinstance` checks the type. Recursion handles lists nested to any depth.

Solution 13.5

Given two lists, find the common elements (intersection) and combined unique elements (union).

```
def compare_lists(a, b):
    set_a, set_b = set(a), set(b)
    common = list(set_a & set_b)
    combined = list(set_a | set_b)
    return common, combined

common, union = compare_lists([1, 2, 3, 4], [3, 4, 5, 6])
print("Common:", sorted(common))
print("Union:", sorted(union))
```

```
Output ▸ Common: [3, 4]
Union: [1, 2, 3, 4, 5, 6]
```

How it works: Converting lists to sets unlocks the `&` (intersection) and `|` (union) operators, which do the heavy lifting in one character each. Sets make these comparisons far cleaner than nested loops.

Solution 13.6

Group a list of words by their first letter into a dictionary.

```
def group_by_first(words):
    groups = {}
    for word in words:
        key = word[0].lower()
        groups.setdefault(key, []).append(word)
    return groups

result = group_by_first(["apple", "avocado", "banana", "cherry", "berry"])
for letter, items in result.items():
    print(letter, "->", items)
```

```
Output ▸ a -> ['apple', 'avocado']
b -> ['banana', 'berry']
c -> ['cherry']
```

How it works: `setdefault(key, [])` returns the existing list for a letter, or creates an empty one if it is the first time we see that letter — then we append. This grouping pattern is extremely common in real data processing.

Chapter 14: Mixed / Interview Warm-up Problems

Solution 14.1

Validate whether a password is strong: at least 8 characters, containing an uppercase letter, a lowercase letter, and a digit.

```
def is_strong(password):
    if len(password) < 8:
        return False
    has_upper = any(c.isupper() for c in password)
    has_lower = any(c.islower() for c in password)
    has_digit = any(c.isdigit() for c in password)
    return has_upper and has_lower and has_digit

for pw in ["abc", "password", "Pass1234"]:
    print(pw, "->", is_strong(pw))
```

```
Output ▸ abc -> False
password -> False
Pass1234 -> True
```

How it works: `any(...)` returns True if at least one character meets a condition. We check length plus three character-type rules and combine them with `and`. This mirrors real signup-form validation.

Solution 14.2

Convert a temperature between Celsius and Fahrenheit based on a unit argument.

```
def convert_temp(value, unit):
    if unit.upper() == "C":
        return value * 9/5 + 32, "F"
    elif unit.upper() == "F":
        return (value - 32) * 5/9, "C"
    else:
        raise ValueError("Unit must be C or F")

result, to_unit = convert_temp(100, "C")
print(f"{result} {to_unit}")
```

```
Output ▸ 212.0 F
```

How it works: Depending on the unit, we apply the correct formula and return both the converted value and its new unit. Raising `ValueError` for an unknown unit combines conditionals with exception handling from Chapter 8.

Solution 14.3

Simulate a simple ATM: given a balance and a list of transactions (positive = deposit, negative = withdrawal), process them and reject withdrawals that exceed the balance.

```
def run_atm(balance, transactions):
    for amount in transactions:
        if amount < 0 and abs(amount) > balance:
            print(f"Declined: withdraw {abs(amount)} (balance {balance})")
            continue
        balance += amount
        action = "Deposited" if amount > 0 else "Withdrew"
        print(f"{action} {abs(amount)} -> balance {balance}")
    return balance

final = run_atm(1000, [500, -200, -2000, -800])
print("Final balance:", final)
```

```
Output ▸ Deposited 500 -> balance 1500
Withdrew 200 -> balance 1300
Declined: withdraw 2000 (balance 1300)
Withdrew 800 -> balance 500
Final balance: 500
```

How it works: We loop through transactions, rejecting a withdrawal only when it exceeds the current balance (using `continue` to skip it). This combines loops, conditionals, and running state — a realistic mini-simulation of the kind asked in interviews.

Solution 14.4

Given a sentence, return the longest word and its length.

```
def longest_word(sentence):
    words = sentence.split()
    longest = max(words, key=len)
    return longest, len(longest)

word, length = longest_word("Python makes programming enjoyable")
print(f"Longest: {word} ({length} letters)")
```

```
Output ▸ Longest: programming (11 letters)
```

How it works: `max(words, key=len)` finds the word with the greatest length — `key=len` tells max to compare by length rather than alphabetically. A clean one-liner that shows off the `key` parameter.

Solution 14.5

Check if a string of brackets is balanced (every opening bracket has a matching closing one).

```
def is_balanced(s):
    stack = []
    pairs = {")": "(", "]": "[", "}": "{"}
    for ch in s:
        if ch in "([{":
            stack.append(ch)
        elif ch in ")]}":
            if not stack or stack.pop() != pairs[ch]:
                return False
    return len(stack) == 0

print(is_balanced("({[]})"))   # True
print(is_balanced("([)]"))     # False
```

```
Output ▸ True
False
```

How it works: We push opening brackets onto a stack (a list). For each closing bracket, we check that the most recent opening bracket matches. If the stack is empty at the end, everything paired up. This is a famous interview question demonstrating the stack concept without needing DSA theory.

Solution 14.6

Format a list of names into a readable string: 'A, B and C'.

```
def format_names(names):
    if not names:
        return ""
    if len(names) == 1:
        return names[0]
    return ", ".join(names[:-1]) + " and " + names[-1]

print(format_names(["Amy", "Bob", "Cara"]))
print(format_names(["Amy", "Bob"]))
print(format_names(["Amy"]))
```

```
Output ▸ Amy, Bob and Cara
Amy and Bob
Amy
```

How it works: We handle edge cases first (empty list, single name). Otherwise we join all but the last name with commas, then attach the last with 'and'. Careful edge-case handling like this is exactly what interviewers watch for.

APPENDIX

Quick Reference & Tips

A. Python Cheat Sheet

String Methods

Method	Does	Method	Does
`.upper()`	UPPERCASE	`.strip()`	trim spaces
`.lower()`	lowercase	`.split()`	str → list
`.replace()`	swap text	`.join()`	list → str
`.find()`	index of	`.count()`	occurrences
`.isdigit()`	all digits?	`.title()`	Title Case

List Methods

Method	Does	Method	Does
`.append()`	add to end	`.pop()`	remove last
`.insert()`	add at index	`.remove()`	delete value
`.sort()`	sort in place	`.reverse()`	flip order
`.index()`	find position	`.extend()`	add many

Dictionary Methods

Method	Does	Method	Does
`.get()`	safe access	`.items()`	key-value pairs
`.keys()`	all keys	`.values()`	all values
`.setdefault()`	get or create	`.update()`	merge dict

Handy One-Liners

```
s[::-1]                     # reverse a string or list
list(set(items))            # remove duplicates
sorted(data, key=len)       # sort by a custom key
[x for x in nums if x > 0]  # filter with comprehension
freq[k] = freq.get(k, 0) + 1  # frequency counter
a, b = b, a                 # swap two variables
max(d, key=d.get)           # dict key with largest value
" ".join(map(str, nums))    # numbers -> spaced string
any(c.isdigit() for c in s)  # does it contain a digit?
n % 2 == 0                  # even number check
```

B. Coding Round Survival Tips

1. **Read the whole question twice.** Note inputs, outputs, and edge cases before typing.
2. **Clarify edge cases:** empty input, negatives, duplicates, single elements. Interviewers award points for spotting these.
3. **Start simple, then optimize.** A working brute-force answer beats an elegant one that does not run.
4. **Think out loud.** Explaining your approach shows reasoning even if the code is imperfect.
5. **Use meaningful names.** `total`, `count`, `result` — never `a`, `b`, `c` for real logic.
6. **Test with examples.** Trace your code with a sample input before declaring it done.
7. **Know your built-ins.** `sorted`, `enumerate`, `zip`, `set`, and comprehensions save time and impress.
8. **Handle errors.** Wrapping risky input parsing in `try/except` shows production awareness.
9. **Keep functions small.** One function, one job — easier to write, test, and explain.
10. **Practice patterns.** Frequency counters, two-pointer scanning, and string manipulation cover a huge share of coding-round questions.

REAL-WORLD USE

Final word: consistency beats intensity. Solving three problems a day for a month builds more fluency than one marathon session. Revisit the problems in Part Four until the patterns feel automatic — that fluency is exactly what a coding round measures. You have got this.